



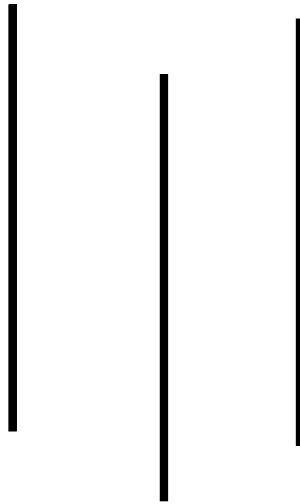
Multiple View Geometry

Lab1 – Camera Calibration

Lab Sessions: 2 (4 hours)

Total Work Load: 10 hours

Programming Language: MATLAB



Submitted by

1. **Pravin Oli** (u1999699) [pravin.oli.08@gmail.com]
2. **Gebrecherkos Gebrezgabhier** (u1999542) [chereg2016@gmail.com]

Table of Contents

1.OBJECTIVE:	3
2.IMPLEMENTATION:	3
PART-1	3
Step 1. Intrinsic parameters and Extrinsic Parameters with the given values.	3
Step 2. Create Camera Intrinsic & Extrinsic matrix.	3
Step 3. Define a set of 3D points in the range [-480:480; -480:480; -480:480].	4
Step 4. Compute the projection of the 3D points on the image plane by using the camera projection matrix.	5
Step 5. Plot the 2D points.	6
Question_1:	7
Step 6. Estimate the 3x4 camera projection matrix P by using the method of Hall.	7
Step 7. Extract the intrinsic parameter matrix K, and the camera rotation matrix R_{wc}	8
PART2	9
Step 8. Extract the intrinsic parameter matrix K, and the camera rotation matrix R_{wc}	9
Question_2:	11
Question_3: Why is the axis skew parameter γ different from zero?	12
Step 9. Compute 2D points and compare with step .compute the average projection error.	12
Step 10. Plot a graph of the average projection error as a function of the number of points	13
Appendix	14
Appendix-I (MATLAB Code)	14
Appendix-II (Intrinsic parameters and the rotation matrix code)	19
Appendix-III (Output values)	20
Appendix-IV (Output Figures)	21

1.OBJECTIVE:

- To gain experience on the basic concepts of camera projection geometry, through the calibration of a simulated camera.

2.IMPLEMENTATION:

PART-1

Step 1. Intrinsic parameters and Extrinsic Parameters with the given values.

→ In this step we will initialize the given intrinsic and extrinsic parameter in MATLAB code

Code →

```
% Step 1: Initializing of intrinsic and extrinsic parameters
% Intrinsic parameters Initializations
au = 557.0943;
av = 712.9824;
u0 = 326.3819;
v0 = 298.6679;

% Location of the world reference frame in camera coordinates in
mm
trans_x = 100;
trans_y = 0;
trans_z = 1500;

% World rotation w.r.t. camera coordinates
% Euler XYZ angles
rot_x_angle = 0.8 * pi / 2;
rot_y_angle = -1.8 * pi / 2;
rot_xl_angle = pi / 5;
```

Step 2. Create Camera Intrinsic & Extrinsic matrix.

→ In this step we will calculate the camera intrinsic and extrinsic matrix and we will calculate camera projection matrix.

Code →

```
% Step 2: Calculating the camera intrinsic and extrinsic
transformation matrix
% Intrinsic matrix
intrinsicMatrix = [au, 0, u0; 0, av, v0; 0, 0, 1];

% Rotation matrices using Euler angles XYZ
rot_x_matrix = [1, 0, 0; 0, cos(rot_x_angle), -sin(rot_x_angle);
0, sin(rot_x_angle), cos(rot_x_angle)];
rot_y_matrix = [cos(rot_y_angle), 0, sin(rot_y_angle); 0, 1, 0; -
sin(rot_y_angle), 0, cos(rot_y_angle)];
rot_xl_matrix = [1, 0, 0; 0, cos(rot_xl_angle), -sin(rot_xl_angle);
0, sin(rot_xl_angle), cos(rot_xl_angle)];

% Combining the rotation matrices
rot_combined_matrix = rot_x_matrix * rot_y_matrix * rot_xl_matrix;

% Creating the rotation matrix in homogeneous form
```

```
rotationMatrix = [rot_combined_matrix, [0; 0; 0]; 0, 0, 0, 1];

% Translation matrix (4x1)
translationMatrix = [trans_x; trans_y; trans_z; 1];

% Combining rotation and translation to form the camera-to-world
transformation matrix
camWorldTransform = [rotationMatrix(:, 1:3), translationMatrix];

% Projection Matrix
projMatrix = intrinsicMatrix * camWorldTransform(1:3, :);
disp('Camera Projection Matrix');
disp(projMatrix);
```

Output →

Camera Projection Matrix =
1.0e+05 *

```
[-0.0050    0.0009   -0.0040    5.4528
 -0.0018    0.0074    0.0015    4.4800
 0.0000    0.0000   -0.0000    0.0150]
```

Step 3. Define a set of 3D points in the range [-480:480; -480:480; -480:480].

→ In this step we will create 6 sets (x,y,z) random 3D point between minimum and maximum value of -480 to 480.

Code →

```
% Step 3: Create 6 random 3D points
% Function to generate 3D points
function random_points = generate_3d_points(min_val, max_val,
num_points)
    random_points = min_val + (max_val - min_val) *
rand(num_points, 3);
end

% Input to function above
min_val = -480;
max_val = 480;
num_points_3d = 6;
points_3d = generate_3d_points(min_val, max_val, num_points_3d);
disp('Random 3D Points:');
disp(points_3d);

% 3D plot of the 3D Points
figure('Name', '3D Points Distribution', 'NumberTitle', 'off');
scatter3(points_3d(:,1), points_3d(:,2), points_3d(:,3), 100,
'filled', 'MarkerFaceColor', 'cyan');
grid on;
xlabel('X (mm)');
ylabel('Y (mm)');
zlabel('Z (mm)');
```

```
title('3D Points Distribution');
axis equal;
hold on;
```

Output →

Random 3D Points:

466.4649	207.2065	432.1089
267.3084	52.9795	-456.8156
433.1138	-106.5251	-162.9049
101.7946	197.1172	-192.7747
-247.4553	267.6601	205.5170
68.8054	-168.4522	-84.5299

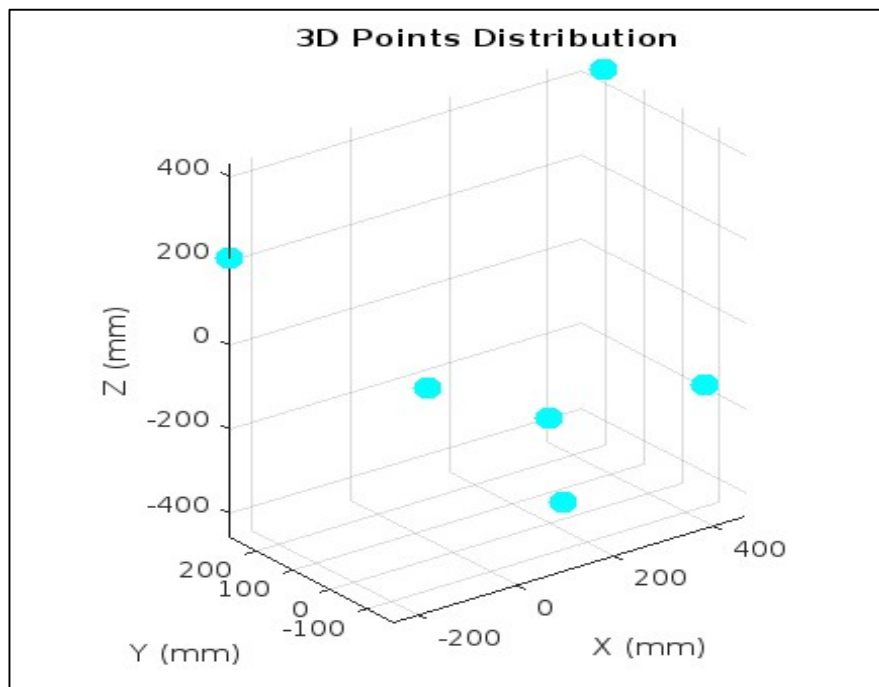


Figure 1(Step-3) 3D point distribution for random points in the range [-480:480; -480:480; -480:480].

Step 4. Compute the projection of the 3D points on the image plane by using the camera projection matrix.

→ In this step we will compute the projection of above 3D points into 2D plane using camera projection matrix.

Code →

```
% Step 4: Project the 3D points onto the image plane
function points_2d = project_3d_to_2d(num_points_3d,
points_3d, projMatrix)
    % Convert 3D points to homogeneous coordinates (add a row
of ones)
    points_3d_homogeneous = [points_3d.'; ones(1,
num_points_3d)];

    % Projecting 3D points onto the image plane using the
projection matrix
```

```

projected_points_homogeneous = projMatrix *
points_3d_homogeneous;

% Normalizing
points_2d = projected_points_homogeneous(1:2, :) ./
projected_points_homogeneous(3, :);
end
points_2d = project_3d_to_2d(num_points_3d, points_3d,
projMatrix);

% Display the projected points
disp('Projection of the 3D points on the 2D image plane:');
disp(points_2d);

```

Output →

Projection of the 3D points on the 2D image plane:

120.4837	311.9858	239.1094	331.2548	415.4323	359.0086
440.0861	191.6141	165.5386	305.9184	489.9512	202.6645

Step 5. Plot the 2D points.

→ In this step we will plot the above 2D points in a scatter plot.

Code →

```

% Step 5: Plot the 2D points in a separate window
figure('Name', 'Projected 2D Points on the Image Plane',
'NumberTitle', 'off');
scatter(points_2d(1, :), points_2d(2, :), 100, 'filled',
'MarkerFaceColor', 'magenta');
grid on;
xlabel('u (pixels)');
ylabel('v (pixels)');
title('2D Projection of 3D Points on the Image Plane');
axis equal;

```

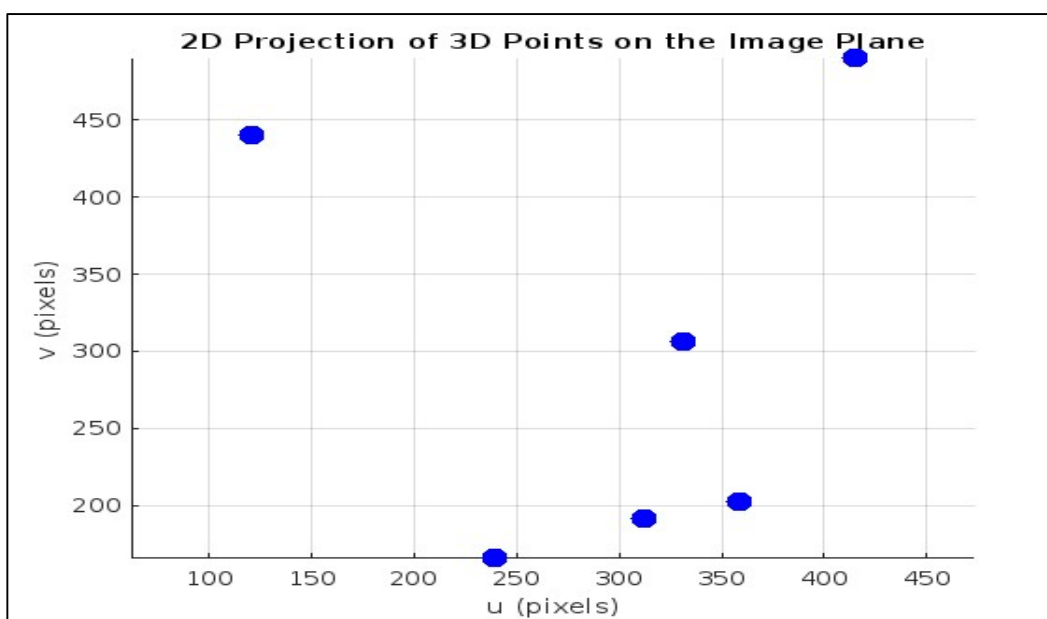


Figure 2 (Step-5) Projection of the 3D points on the 2D image plane

Question_1: Will the distribution of points in the image affect the accuracy in the computation?

→ Yes, the distribution of points in the image affects the accuracy in the computation.

If the 2D points are clustered in a small region of the image (e.g., all near the centre or in a corner), the calibration or projection can become less accurate, it can lead to poor conditioning of the system of equations used to estimate the camera projection matrix, making the solution more sensitive to noise or numerical errors.

A well-distributed set of points across the image plane provides better geometric information for the camera calibration process. If points are concentrated in a small region

Step 6. Estimate the 3x4 camera projection matrix P by using the method of Hall.

→ In this step we will calculate the estimated camera projection matrix using the method of hall by using the 3D points created in Step 3 and their projection obtained in Step 5

Code →

```
% Step 6: Estimating projection matrix using Hall's method
estimated_P = estimate_proj_matrix(points_2d, points_3d,
num_points_3d, projMatrix);
disp('The projection matrix using Hall''s method is:');
disp(estimated_P);

function estimated_P = estimate_proj_matrix(points_2d,
points_3d, num_points, original_P)
    % Initializing the Q matrix and B vector
    Q_matrix = zeros(2 * num_points, 11);
    B_vector = zeros(2 * num_points, 1);

    for i = 1:num_points
        % Building the Q matrix
        Q_matrix(2 * i - 1, :) = [points_3d(i, :), 1, 0, 0,
0, 0, ...
        -points_2d(1, i) * points_3d(i, 1), -points_2d(1,
i) * points_3d(i, 2), -points_2d(1, i) * points_3d(i, 3)];
        Q_matrix(2 * i, :) = [0, 0, 0, 0, points_3d(i, :), 1,
...
        -points_2d(2, i) * points_3d(i, 1), -points_2d(2,
i) * points_3d(i, 2), -points_2d(2, i) * points_3d(i, 3)];

        % Building the B vector (2D image points)
        B_vector(2 * i - 1) = points_2d(1, i); % u-coordinate
        B_vector(2 * i) = points_2d(2, i); % v-coordinate
    end

    % Solving
    A_vector = pinv(Q_matrix) * B_vector;

    % Adding 1 for homogeneous scale
    A_vector = [A_vector; 1];
```

```
% Reshape the vector into a 3x4 projection matrix
estimated_P = reshape(A_vector, 4, 3)';
end
```

Output →

The projection matrix using Hall's method is:

```
-0.3324  0.0624 -0.2662 363.5215
-0.1207  0.4903  0.1028 298.6679
0.0001  0.0004 -0.0005 1.0000
```

Step 7. Extract the intrinsic parameter matrix **K**, and the camera rotation matrix **Rwc**

→ In this step we will compare the projection matrix obtained in Step 6 (by hall method) with the original projection matrix calculated in step 2 using intrinsic and extrinsic parameters.

Code →

```
% Step 7: Compare estimated matrix to the original one and
extract K and R
%Comparing the matrix obtained in Step 6 (estimated_P) to the
one defined in step 2 (projMatrix).
% Difference betn (estimated_P) to the (projMatrix).
matrix_difference = projMatrix - estimated_P;
disp('Difference between Original and Estimated Projection
Matrices:');
disp(matrix_difference);
```

Output →

Difference between Original and Estimated Projection Matrices:

```
1.0e+05 *

-0.0050  0.0009 -0.0040  5.4492
-0.0018  0.0074  0.0015  4.4770
0.0000  0.0000 -0.0000  0.0150]
```

→ After that we will extract the intrinsic and extrinsic parameters from the above calculated projection matrix

Code →

```
% Function to extract intrinsic parameters and rotation
matrix from projection matrix
[K, cRw] = get_intrinsics_from_proj_matrix(estimated_P);
disp('Intrinsic Parameter Matrix K:');
disp(K);
disp('Camera Rotation Matrix R_w to c:');
disp(cRw);

function [K, cRw] = get_intrinsics_from_proj_matrix(P)
% Get left-side 3x3 block of P
M = P(:,1:3);
[Q,R] = qr(rot90(M,3));
R = rot90(R,2)';
Q = rot90(Q);
```

```
% Determinant checking of Q to make cRw
if det(Q) < 0
    cRw = -Q;
else
    cRw = Q;
end

%Normalized intrinsics
K = R / R(3,3);

end
```

Output →

Intrinsic Parameter Matrix K:

557.0943	-0.0000	326.3819
0	712.9824	298.6679
0	0	1.0000

Camera Rotation Matrix R_w to c:

-0.9511	-0.1816	-0.2500
-0.2939	0.7817	0.5501
0.0955	0.5967	-0.7968

PART2

Step 8. Extract the intrinsic parameter matrix K, and the camera rotation matrix Rwc

→ At first Gaussian noise is added to the 2D points. The discrepancies is between the range [-1,+1] pixels for the 95% of points, which means mean noise would be 0 and standard deviation of noise would be 0.5. It is done by using function normrnd.

Code →

```
% Step 8: Adding Gaussian noise to all the 2D points
% Generate Gaussian noise
mean_noise = 0; % mean of the noise
std_noise = 0.5; % standard deviation of the noise for 95% within
±1 pixel
noise = normrnd(mean_noise, std_noise, size(points_2d)); %
Generate noise

% Add noise to the projected 2D points
noisy_points_2d = points_2d + noise;

% Display the noisy projected points
disp('Noisy Projection of the 3D points on the 2D image plane:');
disp(noisy_points_2d);

% Plot the noisy 2D points in a separate window
figure('Name', 'Noisy Projected Points on the Image Plane',
'NumberTitle', 'off');
scatter(noisy_points_2d(1, :), noisy_points_2d(2, :), 100,
'filled', 'MarkerFaceColor', 'red');
grid on;
```

```
xlabel('u (pixels)');
ylabel('v (pixels)');
title('Noisy 2D Projection of 3D Points on the Image Plane');
axis equal;
```

Output →

Noisy Projection of the 3D points on the 2D image plane:

120.7882	311.8688	239.4378	331.5515	416.5003	358.9294
440.2759	191.5483	165.3399	305.6959	489.6568	202.4050

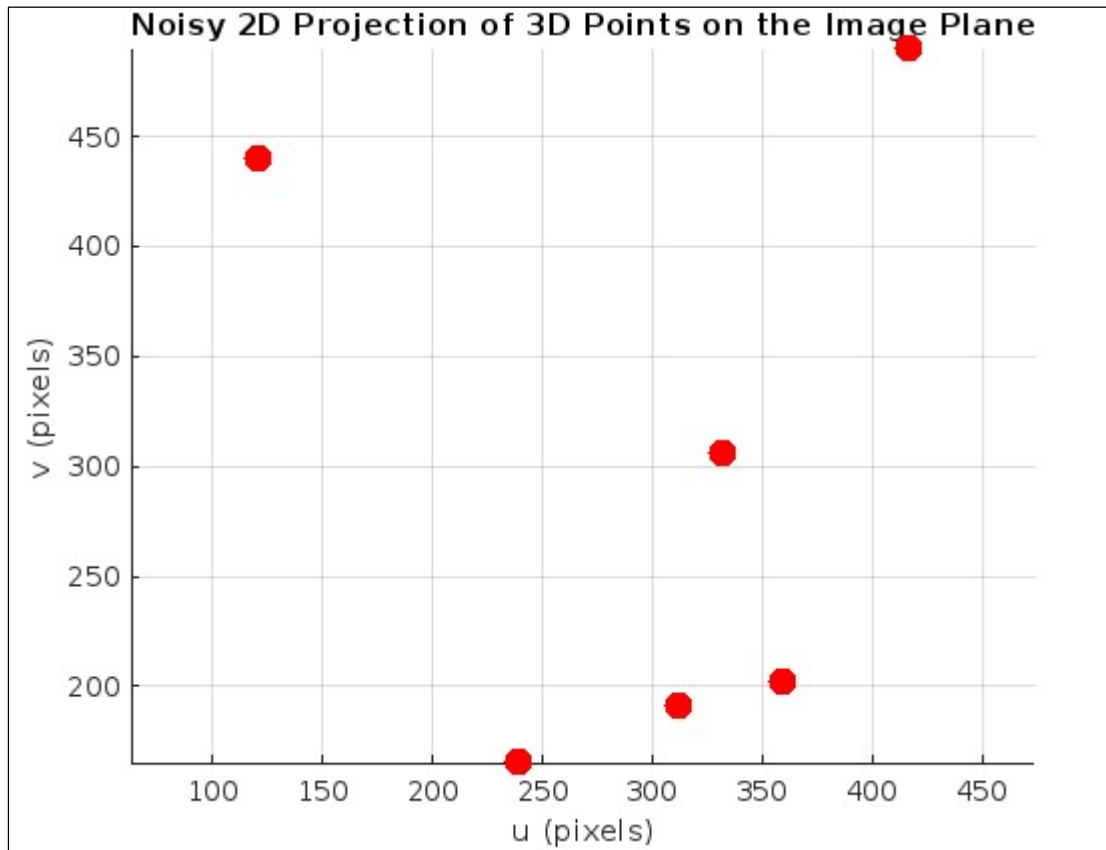


Figure 3 (Step-8a) Noisy Projection of the 3D points on the 2D image plane

After that, we will calculate the projection matrix you obtain with the noise-free points. And at last, the intrinsic parameter matrix K , and the camera rotation matrix R_{wc} from the camera projection is calculated.

⇒

Code →

```
% Repeating Step 6 to Estimate noisy projection matrix using
Hall's method
estimated_P_noisy = estimate_proj_matrix(noisy_points_2d,
points_3d, num_points_3d, projMatrix);
disp('The projection matrix using Hall''s method with noisy points
is:');
disp(estimated_P_noisy);
```

Output →

The projection matrix using Hall's method with noisy points is:

```
-0.3333  0.0570 -0.2649 363.8123
```

-0.1218	0.4846	0.1011	297.9927
0.0001	0.0004	-0.0005	1.0000

Code →

```
%Comparing the projection matrix obtained in step 6, with the
noise-free points.
% Difference betn (estimated_P_noisy) to the (estimated_P).
matrix_difference2 = estimated_P-estimated_P_noisy;
disp('Difference between estimated Projection Matrices without
noisa and with noise:');
disp(matrix_difference2);
```

Output →

Difference between estimated Projection Matrices without noisa and with noise:			
0.0009	0.0053	-0.0013	-0.2908
0.0011	0.0058	0.0017	0.6752
0.0000	0.0000	0.0000	0

Code →

```
%Extracting K and R from the noisy projection matrix
[K_noisy, cRw_noisy] =
get_intrinsics_from_proj_matrix(estimated_P_noisy);

% Display the results
disp('Intrinsic Parameter Matrix K (from noisy projection):');
disp(K_noisy);

disp('Camera Rotation Matrix R_w to c (from noisy projection):');
disp(cRw_noisy);
```

Output →

Intrinsic Parameter Matrix K (from noisy projection):		
564.1877	2.0730	333.9458
0	724.1756	284.5798
0	0	1.0000

Camera Rotation Matrix R_w to c (from noisy projection):		
-0.9526	-0.1898	-0.2377
-0.2912	0.7948	0.5324
0.0878	0.5764	-0.8124

Question 2: How did the intrinsic parameters change?

→ The intrinsic parameters, represented by the matrix K (α_u , α_v are focal length, u_0 , v_0 are principal focal and Skew parameter γ) describe the internal characteristics of camera. When there is noise, intrinsic parameters become less accurate.

$$K = \begin{bmatrix} \alpha_u & \gamma & u_0 \\ 0 & \alpha_v & v_0 \\ 0 & 0 & 1 \end{bmatrix}$$

The focal lengths may change slightly due to the added noise, which can alter the perceived scaling of the 2D points relative to their 3D counterparts.

The position of the principal point may shift according to impact of noise to the estimated centre of the image, affecting alignment with the actual camera optical centre.

Changes in noise can affect skew parameter by possibly increasing its value due to deviations in point distributions., If the skew parameter is non-zero, it suggests a misalignment between the x and y pixel axes.

Question_3: Why is the axis skew parameter γ different from zero?

→ The skew parameter γ indicates the degree to which the axes of the image sensor are not perpendicular to each other.

The axis skew parameter γ different from zero because of many factors:

Addition of Gaussian 2D noise that cause distortion so to avoid them γ different from zero.

Manufacturing tolerances and calibration errors can lead to a non-zero skew parameter.

Step 9. Compute 2D points and compare with step .compute the average projection error.

→ In this step Euclidean distance between the 2D points of step 4 and those of step 8. is calculated for comparison of projection error

Code →

```
% Step 9: Compute the 2D points using noisy estimated projection
matrix and
% compare with original projection matrix
% Project the 3D points using the noisy projection matrix
noisy_proj_matrix = estimated_P_noisy; % Use the projection matrix
from step 8
new_points_2d = project_3d_to_2d(num_points_3d, points_3d,
noisy_proj_matrix);

% Display the newly projected points
disp('Noisy Projection of 3D points on the 2D image plane with
noise:');
disp(noisy_points_2d);

% Euclidean distances between the original 2D points and noisy
projected points
euclidean_distances = sqrt(sum((points_2d - noisy_points_2d).^2,
1));

%Average projection error
average_projection_error = mean(euclidean_distances);
disp('Average Projection Error between noise-free and noisy
points:');
disp(average_projection_error);
```

Output →

Noisy Projection of 3D points on the 2D image plane with noise:

120.7882	311.8688	239.4378	331.5515	416.5003	358.9294
440.2759	191.5483	165.3399	305.6959	489.6568	202.4050

Average Projection Error between noise-free and noisy points:
0.4378

Step 10. Plot a graph of the average projection error as a function of the number of points

➔ In this step number of 3D points is increased up to 50 points. The projection error is calculated for each increment. From the graph We can conclude that more accuracy is obtained if we increase the points.

Code ➔

```
% Step 10: Plot the average projection error as a function of the
% Step 10: Increase the number of 3D points and repeat steps 8 and
9
num_points_array = 1:50;
average_errors = zeros(length(num_points_array), 1);

for idx = 1:length(num_points_array)
    num_points_3d = num_points_array(idx); % Current number of
    points
    points_3d = generate_3d_points(min_val, max_val,
    num_points_3d); % Generate new random 3D points

    % Repeating Step 4: Project the 3D points onto the image plane
    (using original projection matrix)
    points_2d = project_3d_to_2d(num_points_3d, points_3d,
    projMatrix);

    % Repeating Step 8: Add Gaussian noise to the 2D points and
    repeat the projection
    mean_noise = 0; % mean of the noise
    std_noise = 0.5; % standard deviation of the noise for 95%
    within ±1 pixel
    noise = normrnd(mean_noise, std_noise, size(points_2d)); %
    Generate noise

    % Adding noise to the projected 2D points
    noisy_points_2d = points_2d + noise;

    % Repeating Step 6: Projection matrix using noisy 2D points
    estimated_P_noisy = estimate_proj_matrix(noisy_points_2d,
    points_3d, num_points_3d, projMatrix);

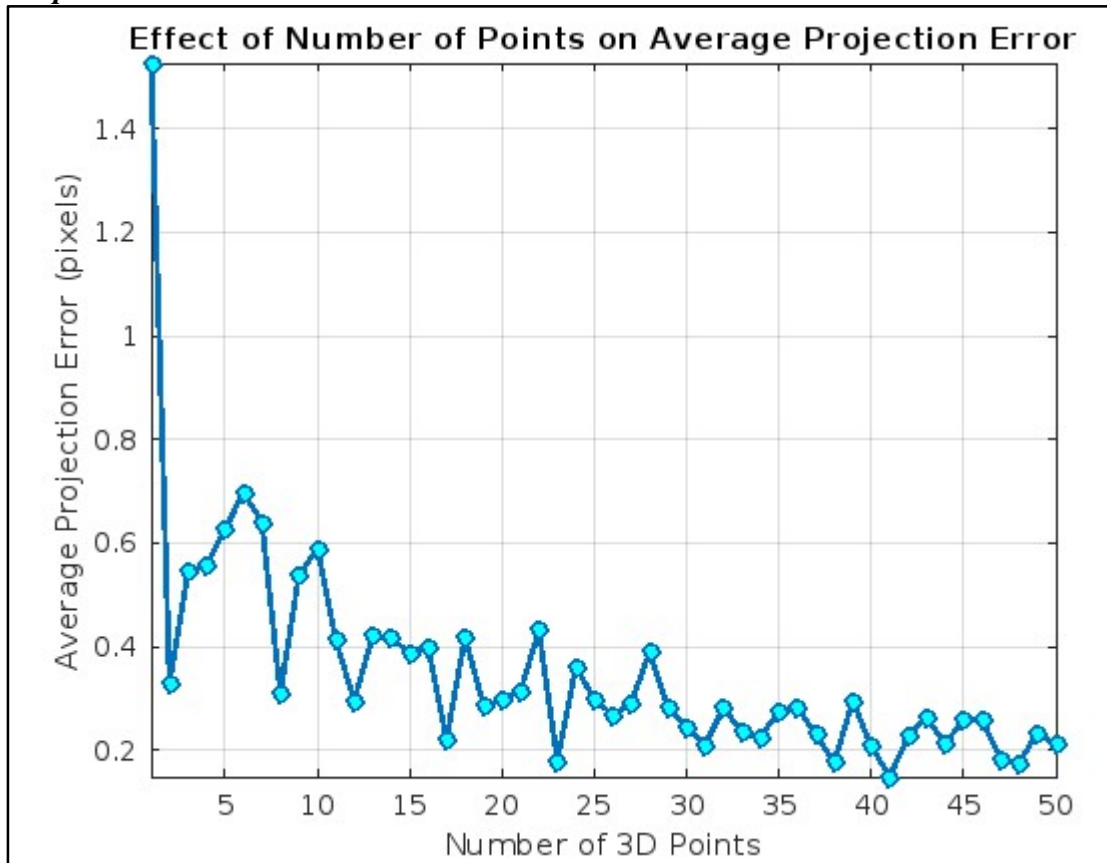
    % Repeating Step 9: New 2D points using the noisy estimated
    projection matrix
    new_points_2d = project_3d_to_2d(num_points_3d, points_3d,
    estimated_P_noisy);

    %Average projection error
    euclidean_distances = sqrt(sum((points_2d - new_points_2d).^2,
    1));
    average_errors(idx) = mean(euclidean_distances);
end

% Step 10: Plotting the average projection error as a function of
the number of points
```

```
figure('Name', 'Average Projection Error vs Number of Points',
'NumberTitle', 'off');
plot(num_points_array, average_errors, '-o', 'MarkerFaceColor',
'cyan', 'LineWidth', 2);
grid on;
xlabel('Number of 3D Points');
ylabel('Average Projection Error (pixels)');
title('Effect of Number of Points on Average Projection Error');
axis tight;
```

Output →



Appendix

Appendix-I (MATLAB Code)

```
% Step 1: Initializing of intrinsic and extrinsic parameters
% Intrinsic parameters Initializations
au = 557.0943;
av = 712.9824;
u0 = 326.3819;
v0 = 298.6679;

% Location of the world reference frame in camera coordinates in mm
trans_x = 100;
trans_y = 0;
```

```

trans_z = 1500;

% World rotation w.r.t. camera coordinates
% Euler XYZ angles
rot_x_angle = 0.8 * pi / 2;
rot_y_angle = -1.8 * pi / 2;
rot_x1_angle = pi / 5;

% Step 2: Calculating the camera intrinsic and extrinsic transformation matrix
% Intrinsic matrix
intrinsicMatrix = [au, 0, u0; 0, av, v0; 0, 0, 1];

% Rotation matrices using Euler angles XYZ
rot_x_matrix = [1, 0, 0; 0, cos(rot_x_angle), -sin(rot_x_angle); 0, sin(rot_x_angle), cos(rot_x_angle)];
rot_y_matrix = [cos(rot_y_angle), 0, sin(rot_y_angle); 0, 1, 0; -sin(rot_y_angle), 0, cos(rot_y_angle)];
rot_x1_matrix = [1, 0, 0; 0, cos(rot_x1_angle), -sin(rot_x1_angle); 0, sin(rot_x1_angle), cos(rot_x1_angle)];

% Combining the rotation matrices
rot_combined_matrix = rot_x_matrix * rot_y_matrix * rot_x1_matrix;

% Creating the rotation matrix in homogeneous form
rotationMatrix = [rot_combined_matrix, [0; 0; 0]; 0, 0, 0, 1];

% Translation matrix (4x1)
translationMatrix = [trans_x; trans_y; trans_z; 1];

% Combining rotation and translation to form the camera-to-world transformation matrix
camWorldTransform = [rotationMatrix(:, 1:3), translationMatrix];

% Projection Matrix
projMatrix = intrinsicMatrix * camWorldTransform(1:3, :);
disp('Camera Projection Matrix');
disp(projMatrix);

% Step 3: Create 6 random 3D points
% Function to generate 3D points
function random_points = generate_3d_points(min_val, max_val, num_points)
    random_points = min_val + (max_val - min_val) * rand(num_points, 3);
end

% Input to function above
min_val = -480;
max_val = 480;
num_points_3d = 6;
points_3d = generate_3d_points(min_val, max_val, num_points_3d);
disp('Random 3D Points:');
disp(points_3d);

% 3D plot of the 3D Points
figure('Name', '3D Points Distribution', 'NumberTitle', 'off');
scatter3(points_3d(:,1), points_3d(:,2), points_3d(:,3), 100, 'filled', 'MarkerFaceColor', 'cyan');
grid on;
xlabel('X (mm)');
ylabel('Y (mm)');

```

```

xlabel('Z (mm)');
title('3D Points Distribution');
axis equal;
hold on;

% Step 4: Project the 3D points onto the image plane
function points_2d = project_3d_to_2d(num_points_3d, points_3d, projMatrix)
    % Convert 3D points to homogeneous coordinates (add a row of ones)
    points_3d_homogeneous = [points_3d.'; ones(1, num_points_3d)];

    % Projecting 3D points onto the image plane using the projection matrix
    projected_points_homogeneous = projMatrix * points_3d_homogeneous;

    % Normalizing
    points_2d = projected_points_homogeneous(1:2, :) ./
projected_points_homogeneous(3, :);
end
points_2d = project_3d_to_2d(num_points_3d, points_3d, projMatrix);

% Display the projected points
disp('Projection of the 3D points on the 2D image plane:');
disp(points_2d);

% Step 5: Plot the 2D points in a separate window
figure('Name', 'Projected 2D Points on the Image Plane', 'NumberTitle', 'off');
scatter(points_2d(1, :), points_2d(2, :), 100, 'filled', 'MarkerFaceColor',
'blue');
grid on;
xlabel('u (pixels)');
ylabel('v (pixels)');
title('2D Projection of 3D Points on the Image Plane');
axis equal;

% Step 6: Estimating projection matrix using Hall's method
estimated_P = estimate_proj_matrix(points_2d, points_3d, num_points_3d,
projMatrix);
disp('The projection matrix using Hall''s method is:');
disp(estimated_P);

function estimated_P = estimate_proj_matrix(points_2d, points_3d, num_points,
original_P)
    % Initializing the Q matrix and B vector
    Q_matrix = zeros(2 * num_points, 11);
    B_vector = zeros(2 * num_points, 1);

    for i = 1:num_points
        % Building the Q matrix
        Q_matrix(2 * i - 1, :) = [points_3d(i, :), 1, 0, 0, 0, 0, ...
-points_2d(1, i) * points_3d(i, 1), -points_2d(1, i) * points_3d(i, 2),
-points_2d(1, i) * points_3d(i, 3)];
        Q_matrix(2 * i, :) = [0, 0, 0, 0, points_3d(i, :), 1, ...
-points_2d(2, i) * points_3d(i, 1), -points_2d(2, i) * points_3d(i, 2),
-points_2d(2, i) * points_3d(i, 3)];

        % Building the B vector (2D image points)
        B_vector(2 * i - 1) = points_2d(1, i); % u-coordinate
        B_vector(2 * i) = points_2d(2, i); % v-coordinate
    end
end

```

```

% Solving
A_vector = pinv(Q_matrix) * B_vector;

% Adding 1 for homogeneous scale
A_vector = [A_vector; 1];

% Reshape the vector into a 3x4 projection matrix
estimated_P = reshape(A_vector, 4, 3)';
end

% Step 7: Compare estimated matrix to the original one and extract K and R
%Comparing the matrix obtained in Step 6 (estimated_P) to the one defined in step
2 (projMatrix).
% Difference betn (estimated_P) to the (projMatrix).
matrix_difference = projMatrix - estimated_P;
disp('Difference between Original and Estimated Projection Matrices:');
disp(matrix_difference);

% Function to extract intrinsic parameters and rotation matrix from projection
matrix
[K, cRw] = get_intrinsics_from_proj_matrix(estimated_P);
disp('Intrinsic Parameter Matrix K:');
disp(K);
disp('Camera Rotation Matrix R_w to c:');
disp(cRw);

function [K, cRw] = get_intrinsics_from_proj_matrix(P)
% Get left-side 3x3 block of P
M = P(:,1:3);
[Q,R] = qr(rot90(M,3));
R = rot90(R,2)';
Q = rot90(Q);

% Determinant checking of Q to make cRw
if det(Q) < 0
    cRw = -Q;
else
    cRw = Q;
end

%Normalized intrinsics
K = R / R(3,3);
end

% Step 8: Adding Gaussian noise to all the 2D points
% Generate Gaussian noise
mean_noise = 0; % mean of the noise
std_noise = 0.5; % standard deviation of the noise for 95% within ±1 pixel
noise = normrnd(mean_noise, std_noise, size(points_2d)); % Generate noise

% Add noise to the projected 2D points
noisy_points_2d = points_2d + noise;

% Display the noisy projected points
disp('Noisy Projection of the 3D points on the 2D image plane:');
disp(noisy_points_2d);

% Plot the noisy 2D points in a separate window

```

```
figure('Name', 'Noisy Projected 2D Points on the Image Plane', 'NumberTitle',
'off');
scatter(noisy_points_2d(1, :), noisy_points_2d(2, :), 100, 'filled',
'MarkerFaceColor', 'red');
grid on;
xlabel('u (pixels)');
ylabel('v (pixels)');
title('Noisy 2D Projection of 3D Points on the Image Plane');
axis equal;

% Repeating Step 6 to Estimate noisy projection matrix using Hall's method
estimated_P_noisy = estimate_proj_matrix(noisy_points_2d, points_3d, num_points_3d,
projMatrix);
disp('The projection matrix using Hall''s method with noisy points is:');
disp(estimated_P_noisy);

%Comparing the projection matrix obtained in step 6, with the noise-free points.
% Difference betn (estimated_P_noisy) to the (estimated_P).
matrix_difference2 = estimated_P-estimated_P_noisy;
disp('Difference between estimated Projection Matrices without noisa and with
noise:');
disp(matrix_difference2);

%Extracting K and R from the noisy projection matrix
[K_noisy, cRw_noisy] = get_intrinsics_from_proj_matrix(estimated_P_noisy);

% Display the results
disp('Intrinsic Parameter Matrix K (from noisy projection):');
disp(K_noisy);

disp('Camera Rotation Matrix R_w to c (from noisy projection):');
disp(cRw_noisy);

% Step 9: Compute the 2D points using noisy estimated projection matrix and
% compare with orginal projection matrix
% Project the 3D points using the noisy projection matrix
noisy_proj_matrix = estimated_P_noisy; % Use the projection matrix from step 8
new_points_2d = project_3d_to_2d(num_points_3d, points_3d, noisy_proj_matrix);

% Display the newly projected points
disp('Noisy Projection of 3D points on the 2D image plane with noise:');
disp(noisy_points_2d);

% Euclidean distances between the original 2D points and noisy projected points
euclidean_distances = sqrt(sum((points_2d - noisy_points_2d).^2, 1));

%Average projection error
average_projection_error = mean(euclidean_distances);
disp('Average Projection Error between noise-free and noisy points:');
disp(average_projection_error);

% Step 10: Increase the number of 3D points and repeat steps 8 and 9
num_points_array = 1:50;
average_errors = zeros(length(num_points_array), 1);

for idx = 1:length(num_points_array)
    num_points_3d = num_points_array(idx); % Current number of points
```

```

points_3d = generate_3d_points(min_val, max_val, num_points_3d); % Generate new
random 3D points

% Repeating Step 4: Project the 3D points onto the image plane (using original
projection matrix)
points_2d = project_3d_to_2d(num_points_3d, points_3d, projMatrix);

% Repeating Step 8: Add Gaussian noise to the 2D points and repeat the projection
mean_noise = 0; % mean of the noise
std_noise = 0.5; % standard deviation of the noise for 95% within ±1 pixel
noise = normrnd(mean_noise, std_noise, size(points_2d)); % Generate noise

% Adding noise to the projected 2D points
noisy_points_2d = points_2d + noise;

% Repeating Step 6: Projection matrix using noisy 2D points
estimated_P_noisy = estimate_proj_matrix(noisy_points_2d, points_3d,
num_points_3d, projMatrix);

% Repeating Step 9: New 2D points using the noisy estimated projection matrix
new_points_2d = project_3d_to_2d(num_points_3d, points_3d, estimated_P_noisy);

%Average projection error
euclidean_distances = sqrt(sum((points_2d - new_points_2d).^2, 1));
average_errors(idx) = mean(euclidean_distances);
end

% Step 10: Plotting the average projection error as a function of the number of
points
figure('Name', 'Average Projection Error vs Number of Points', 'NumberTitle',
'off');
plot(num_points_array, average_errors, '-o', 'MarkerFaceColor', 'cyan',
'LineWidth', 2);
grid on;
xlabel('Number of 3D Points');
ylabel('Average Projection Error (pixels)');
title('Effect of Number of Points on Average Projection Error');
axis tight;

```

Appendix-II (Intrinsic parameters and the rotation matrix code)

The following function allows to extract the intrinsic parameters and the rotation matrix from a projection matrix. It uses the RQ decomposition of a square matrix $M = R \cdot Q$, where R is a upper triangular matrix and Q is an orthogonal matrix.

```

function [K,cRw] = get_intrinsics_from_proj_matrix(P)
% Get left-side 3x3 block of P
M = P(:,1:3);
% This implements the RQ decomposition from QR in matlab
[Q,R] = qr(rot90(M,3));
R = rot90(R,2)';
Q = rot90(Q);
% Check the determinant of Q to make cRw a proper rotation
if det(Q) < 0
cRw = -Q;
else
cRw = Q;
end
% Get the normalized intrinsics

```

$K = R/R(3,3);$

Appendix-III (Output values)

Camera Projection Matrix

1.0e+05 *

-0.0050	0.0009	-0.0040	5.4528
-0.0018	0.0074	0.0015	4.4800
0.0000	0.0000	-0.0000	0.0150

Random 3D Points:

466.4649	207.2065	432.1089
267.3084	52.9795	-456.8156
433.1138	-106.5251	-162.9049
101.7946	197.1172	-192.7747
-247.4553	267.6601	205.5170
68.8054	-168.4522	-84.5299

Projection of the 3D points on the 2D image plane:

120.4837	311.9858	239.1094	331.2548	415.4323	359.0086
440.0861	191.6141	165.5386	305.9184	489.9512	202.6645

The projection matrix using Hall's method is:

-0.3324	0.0624	-0.2662	363.5215
-0.1207	0.4903	0.1028	298.6679
0.0001	0.0004	-0.0005	1.0000

Difference between Original and Estimated Projection Matrices:

1.0e+05 *

-0.0050	0.0009	-0.0040	5.4492
-0.0018	0.0074	0.0015	4.4770
0.0000	0.0000	-0.0000	0.0150

Intrinsic Parameter Matrix K:

557.0943	-0.0000	326.3819
0	712.9824	298.6679
0	0	1.0000

Camera Rotation Matrix R_w to c:

-0.9511	-0.1816	-0.2500
-0.2939	0.7817	0.5501
0.0955	0.5967	-0.7968

Noisy Projection of the 3D points on the 2D image plane:

120.7882	311.8688	239.4378	331.5515	416.5003	358.9294
440.2759	191.5483	165.3399	305.6959	489.6568	202.4050

The projection matrix using Hall's method with noisy points is:

```
-0.3333  0.0570 -0.2649  363.8123
-0.1218  0.4846  0.1011  297.9927
0.0001  0.0004 -0.0005  1.0000
```

Difference between estimated Projection Matrices without noisa and with noise:

```
0.0009  0.0053 -0.0013 -0.2908
0.0011  0.0058  0.0017  0.6752
0.0000  0.0000  0.0000    0
```

Intrinsic Parameter Matrix K (from noisy projection):

```
564.1877    2.0730    333.9458
0          724.1756    284.5798
0           0          1.0000
```

Camera Rotation Matrix R_w to c (from noisy projection):

```
-0.9526 -0.1898 -0.2377
-0.2912  0.7948  0.5324
0.0878  0.5764 -0.8124
```

Noisy Projection of 3D points on the 2D image plane with noise:

```
120.7882    311.8688    239.4378    331.5515    416.5003    358.9294
440.2759    191.5483    165.3399    305.6959    489.6568    202.4050
```

Average Projection Error between noise-free and noisy points:

```
0.4378
```

Appendix-IV (Output Figures)

